ACTIVIDAD INDIVIDUAL 1 · DESARROLLO DE INTERFACES · 0488

Nexo

Sistema multiplataforma para la gestión de comunidades

MEMORIA TÉCNICA, FUNCIONAL Y MANUAL DE USUARIO
Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)
Ionic · Angular · Capacitor · Supabase

Autor: Jorge Berenguer

Fecha de revisión: 31 de octubre de 2026

Repositorio público: https://github.com/jorgeberenguer2023-ops/RetoSupabase1

ESTADO DE ENTREGA: documentación adaptada a la actividad. Antes de entregar faltan evidencias externas que no se deben inventar: URL de producción verificada, usuarios de prueba, informe Lighthouse y evidencias de Git Flow. También está pendiente el origen NoSQL exigido en la fase de datos.

Finalidad

Este documento describe lo que está implementado en el repositorio y contrasta sus funciones con las fases RA 1–7 de la actividad. Incluye una guía de uso y una lista explícita de evidencias pendientes. No afirma como realizados un despliegue, una auditoría de accesibilidad ni pruebas contra servicios remotos que no han sido verificadas.

Enlaces del proyecto

Código: https://github.com/jorgeberenguer2023-ops/RetoSupabase1
Guía y configuración: README.md del repositorio
Documentación generada: documentation/

1. Resumen y arquitectura

Nexo es una aplicación de gestión para comunidades residenciales. El panel administrativo permite mantener registros y consultar indicadores; cada cuenta estándar obtiene una ficha privada con datos de su comunidad. La interfaz se construye con Angular e Ionic y se empaqueta para Android con Capacitor. Supabase proporciona autenticación, PostgreSQL, almacenamiento de imágenes y cambios en tiempo real.

Objetivos funcionales

- Autenticar perfiles con roles de administrador y usuario estándar.
- Crear, consultar, editar y eliminar comunidades con controles de validación.
- Vincular una cuenta a cada comunidad y limitar el acceso mediante políticas del servidor.
- Mostrar métricas y exportar informes CSV/PDF; gestionar fotos en un bucket privado.
- Ofrecer una interfaz adaptable a móvil y Android, con preferencia de modo oscuro.

Arquitectura por capas

Capa	Responsabilidad e implementación
Presentación	Páginas standalone de Angular, plantillas declarativas e Ionic UI; componentes reutilizables para el listado y los indicadores.
Estado y lógica cliente	Servicios Angular tipados con signals y operaciones asíncronas. SupabaseService gestiona autenticación/perfil; CommunitiesService gestiona CRUD, fotos firmadas y Realtime.
Persistencia y autorización	Supabase Auth + PostgreSQL/PostgREST. RLS decide qué filas puede consultar o modificar cada sesión; el navegador no es la frontera de seguridad.
Fotos y cuentas	Supabase Storage privado para imágenes. Edge Function create-community-user invita/vincula cuentas usando secretos gestionados en el servidor.
Aplicación móvil	Capacitor 8; Android package id es.proyecto.nexocomunidades. Filesystem y Share habilitan el flujo nativo de informes.

Modelo de datos

Tabla/objeto	Datos principales y propósito
public.comunidad	id, created_at, comunidad, numero, email, coste_mensual, junta, cancelada, user_id y portal_image_path.
public.profiles	id asociado a auth.users, full_name, role (admin/user) y community_id.
Storage community-photos	Bucket privado; archivos por ID de comunidad, formatos JPG/PNG/WEBP y límite de 5 MB. Se guarda la ruta en comunidad y se generan URL firmadas.

Origen de datos: el proyecto usa una base relacional PostgreSQL. Supabase Realtime notifica cambios de PostgreSQL, pero no es una segunda fuente NoSQL. La integración dual SQL/NoSQL solicitada en la actividad no está implementada y debe añadirse o justificarse con el docente.

2. Fases 1 y 2 · RA 1, RA 2 y RA 3

RA 1 · Enrutamiento y control de acceso

- Las vistas principales se cargan de forma diferida con loadComponent: login, shell, dashboard y comunidades.
- authGuard exige sesión y perfil; loginGuard redirige una sesión activa; adminGuard reserva el dashboard para administradores.
- La seguridad se refuerza en PostgreSQL con RLS. El usuario estándar solo consulta la comunidad asociada; el administrador gestiona los registros.
- El perfil y la sesión se mantienen a través de Supabase Auth; la interfaz representa el estado con signals.

RA 2 · Plantillas, binding y formularios

- Las plantillas Angular expresan las vistas; [(ngModel)] enlaza campos de formularios y los bindings unidireccionales presentan perfil, registros y estados.
- El formulario de comunidades comprueba campos obligatorios, tipos, rangos y correo. emailExists consulta Supabase de forma asíncrona y excluye el registro editado.
- El login y las operaciones de guardado indican carga/errores; la duplicidad de correo se muestra junto al formulario.

RA 3 · Componentes reutilizables

Componente	Entradas y salida	Uso
RecordTableComponent	@Input communities/canManage; @Output edit/remove	Listado con búsqueda por comunidad/correo, filtro de estado y acciones emitidas al padre.
StatCardComponent	@Input label/value/detail/icon	Tarjeta independiente para mostrar un indicador y su explicación visual.

La pantalla principal administra el ciclo de vida del gráfico: lo crea después de renderizar la vista y lo destruye al abandonar el componente. RecordTableComponent delega al padre las acciones de edición y eliminación, en vez de realizar directamente operaciones de base de datos.

Correspondencia con la actividad

El código evidencia rutas protegidas, carga diferida, roles, data binding, validación asíncrona y dos componentes visuales. La calificación final depende de la revisión docente y no se deduce únicamente de esta memoria.

3. Fase 3 · RA 5 · Datos, métricas e informes

Consumo asíncrono y concurrencia

Los servicios usan async/await (Promises) con Supabase. La carga de registros obtiene URL firmadas de las imágenes en paralelo con Promise.all. Las operaciones de carga y guardado propagan errores al componente, que muestra avisos al usuario. El canal Realtime escucha cambios en comunidad y vuelve a cargar los datos cuando llegan eventos.

Dashboard administrativo

- Métricas: total de comunidades, ingresos mensuales activos, activas y canceladas.
- Cálculo de la próxima junta futura entre las comunidades activas.
- Gráfico de barras con Chart.js para comparar las cuotas por comunidad.
- Búsqueda y filtros de estado en el componente de directorio.

Exportación de informes

ReportService genera CSV con separador punto y coma, codificación UTF-8 y protección básica frente a fórmulas de hoja de cálculo. Genera PDF con jsPDF. En navegador inicia descarga; en Android escribe el informe en el caché privado y abre el diálogo nativo de compartir mediante Capacitor Share.

Esquema de flujo

Acción	Ruta de datos
Login	Angular → Supabase Auth → perfil/rol → guards y navegación.
Consulta	Página → CommunitiesService → PostgREST/PostgreSQL con RLS → modelo Community.
Foto	Storage privado → URL firmada temporal → imagen de la comunidad autorizada.
Cambio remoto	PostgreSQL Realtime → señal del servicio → recarga de datos y actualización visual.

Limitación frente al requisito

La actividad pide simular/consumir dos orígenes (SQL y NoSQL/documentos). El código revisado usa PostgreSQL para los datos y Realtime sobre la misma base; no se encontró integración Firestore u otro almacén NoSQL. No se debe presentar Realtime como NoSQL. Para cubrir el criterio, añadir una fuente NoSQL de demostración con responsabilidades claras y manejo de errores/carga, o solicitar al docente una equivalencia aceptada.

4. Fase 4 · RA 4 · Usabilidad y accesibilidad

Diseño adaptable y uso móvil

Las páginas utilizan CSS Grid/Flexbox, límites de ancho y puntos de ruptura para reorganizar cabeceras, fichas y columnas. En móvil, la ficha de comunidad pasa a una columna y se ajustan espaciados y botones. La app se compila como proyecto Android Capacitor; la compilación local valida el ensamblado, no sustituye una prueba en dispositivo.

Semántica y ayudas de acceso observadas

- Estructura HTML con main, section, article, encabezados, tablas con caption y scope en columnas/filas.
- Etiquetas accesibles en controles como login, menú, acciones de edición/borrado y formulario modal; mensajes de error con role="alert" y avisos con role="status".
- Iconos decorativos ocultos a lectores cuando corresponde; imagen del portal con texto alternativo descriptivo.
- Controles Ionic permiten interacción por teclado; el tema oscuro puede activarse desde login y menú y se recuerda en el dispositivo.

WCAG y Lighthouse: evidencia pendiente

El proyecto incluye señales favorables de accesibilidad, pero no se ha ejecutado una auditoría Lighthouse contra una URL pública y no hay puntuaciones verificadas para Accessibility ni Best Practices. Por tanto, no se declara conformidad WCAG ni se afirma el mínimo requerido de 90/100. La rúbrica reserva la valoración máxima para resultados superiores a 95 y navegación por tabulación; también debe verificarse manualmente el contraste de ambos temas, el foco visible, teclado y lector de pantalla.

Procedimiento para completar evidencia

- Desplegar primero una versión accesible desde navegador y probar login/roles.
- Ejecutar Lighthouse en Chrome DevTools (modo móvil y escritorio) sobre páginas representativas; guardar el informe/capturas con URL, fecha y versión.
- Anotar puntuaciones exactas de Accessibility y Best Practices y adjuntar la exportación al paquete de entrega. No editar las puntuaciones ni usar resultados locales como si fueran producción.
- Corregir los hallazgos y repetir la auditoría. Complementar con tabulación sin ratón, foco visible, zoom y lector de pantalla.

Requisito de accesibilidad de la actividad: adjuntar evidencia real con al menos 90/100 en Accessibility y Best Practices. La evidencia aún no está incluida en esta memoria.

5. Manual de usuario

Acceso

- Abrir la URL de producción una vez que esté configurada (pendiente de completar en esta entrega) o instalar la app Android de depuración desde Android Studio.
- Introducir el correo y la contraseña de una cuenta de prueba de Supabase. Las credenciales de evaluación deben crearse solo para el entorno de demostración y proporcionarse de forma controlada; no reutilizar contraseñas personales.
- El administrador accede al resumen; una cuenta estándar entra en Comunidades. Si un usuario aún no está vinculado, la app informa que debe contactar con administración.

Tareas del administrador

- En Comunidades, buscar por nombre/correo y filtrar activas/canceladas.
- Crear una comunidad completando nombre, número de vecinos, correo, cuota, junta y estado. El sistema valida el correo contra los registros existentes e intenta aprovisionar el acceso asociado.
- Editar un registro para cambiar los atributos, asignar/sincronizar accesos o gestionar la fotografía JPG/PNG/WEBP (máximo 5 MB).
- Eliminar una comunidad solo después de confirmar el diálogo.
- En Resumen, revisar indicadores/gráfico y exportar CSV o PDF.

Tareas del usuario estándar

- Abrir Comunidades para ver solo la comunidad vinculada a la cuenta.
- Consultar la fotografía del portal, número de vecinos, cuota mensual, próxima junta, contacto y estado.
- Usar CSV/PDF para exportar los datos accesibles a la propia sesión.
- Cambiar entre modo claro y oscuro; la preferencia queda guardada en el dispositivo.

Uso en Android

Para instalación de desarrollo, usar un equipo con Android Studio, Android SDK y JDK 21. En el repositorio: ejecutar npm ci, después npm run android:sync y npm run android:open; seleccionar un emulador/teléfono y pulsar Run. El uso del backend requiere conexión a Internet. Para distribución, configurar firma release; no compartir ni versionar el keystore.

Ayuda ante problemas

Síntoma	Comprobación inicial
No aparecen datos	Revisar sesión, conexión, URL/clave publicable del entorno, migraciones y RLS de Supabase.
No se crea la cuenta invitada	Desplegar create-community-user; comprobar secrets, SMTP/plantillas/URLs de Auth y permisos de administrador.
No aparece la foto	Aplicar migración de Storage, revisar bucket privado/policies, ruta portal_image_path, tipo/tamaño y vigencia del enlace firmado.
Error al compartir informe en móvil	Verificar sincronización de Capacitor/plugins, permisos y diálogo de compartir del sistema.

6. Fases 5 y 6 · RA 6 y RA 7

RA 6 · Documentación y mantenimiento

El repositorio incluye README con instalación, configuración de Supabase, funcionalidad, Android y comandos. El script npm run docs genera documentación técnica con Compodoc en documentation/. Esta memoria incorpora una guía de usuario y una matriz de evidencias, pero es necesario regenerar Compodoc tras cambios relevantes y entregar una versión del manual junto al PDF exigido.

RA 7 · Control de versiones, CI y despliegue

Aspecto	Estado comprobado / limitación
Repositorio	RetoSupabase1 es público y contiene el proyecto en la raíz.
Historial y ramas	La publicación inicial verificada es un commit en main. No hay evidencia en este historial publicado de ramas develop/feature ni Pull Requests. La actividad pide Git Flow; crear ramas y commits reales no retrospectivos para demostrar evolución.
Integración continua	CI en GitHub Actions ejecuta npm ci, lint, pruebas y build para push/PR en main y develop. No configura despliegue a producción ni pruebas E2E.
Vercel	Existe vercel.json para el fallback SPA, pero un archivo de configuración no confirma que la aplicación esté desplegada. URL de producción: PENDIENTE DE PROPORCIONAR Y VERIFICAR.
Android	Se verificó localmente la compilación Gradle assembleDebug. No se verificó instalación en un dispositivo real ni APK firmada para distribución.

Verificación técnica registrada

Comando/validación	Resultado registrado
npm run build	Correcto; compilación Angular a www. Se observaron advertencias CommonJS de dependencias de jsPDF/canvg.
npm test -- --watch=false	3 archivos de pruebas, 4 pruebas satisfactorias.
npm run lint	Correcto.
npm run android:sync	Correcto; web assets y plugins sincronizados.
Gradle assembleDebug	BUILD SUCCESSFUL con SDK Android local.

Estos resultados verifican el código en este entorno en la fecha de revisión. No acreditan la disponibilidad de Supabase remoto ni una prueba de aceptación en producción.

7. Matriz de requisitos y pendientes de entrega

Resumen de cobertura basada en la inspección del código y las evidencias disponibles. “Parcial/pendiente” significa que falta verificación o una característica expresamente solicitada; no constituye una nota académica.

Criterio	Evidencia actual	Estado / acción
RA 1 · Arquitectura y rutas	Rutas lazy, guards por sesión/rol, perfiles y RLS.	Implementado; probar permisos con dos cuentas reales.
RA 2 · Formularios/binding	Plantillas, ngModel y comprobación async del correo.	Implementado; incluir evidencia funcional.
RA 3 · Componentes	RecordTableComponent y StatCardComponent con Inputs/Outputs.	Implementado.
RA 4 · WCAG/responsive	Diseño adaptable, etiquetas semánticas/ARIA y dark mode.	Parcial: falta Lighthouse real (mínimo 90/100) y pruebas manuales WCAG.
RA 5 · Datos/informes	PostgreSQL async, Realtime, Chart.js, PDF y CSV.	Parcial: falta un origen NoSQL real/mock explícito.
RA 6 · Documentación	README, Compodoc y este manual integrado en la memoria.	Parcial: regenerar docs; entregar una copia única del PDF requerido.
RA 7 · Git/despliegue	Repo público; CI valida lint/tests/build; Vercel SPA config.	Parcial/crítico: faltan historial Git Flow, URL pública funcional y despliegue automatizado.

Campos requeridos antes de enviar el ZIP

Evidencia obligatoria en el PDF	Completar con información comprobada
URL pública en producción	PENDIENTE — no se ha verificado un despliegue.
Usuario de prueba administrador	PENDIENTE — crear cuenta de demostración; facilitar por canal seguro.
Contraseña de prueba administrador	PENDIENTE — contraseña temporal de demostración; no reutilizar secretos personales.
Usuario de prueba estándar	PENDIENTE — crear/vincular a una comunidad sintética.
Contraseña de prueba estándar	PENDIENTE — contraseña temporal de demostración; no publicar credenciales reales.
Lighthouse Accessibility / Best Practices	PENDIENTE — ejecutar en la URL publicada y adjuntar informe/captura real.
Historial Git Flow / ramas	PENDIENTE — conservar evidencia de main, develop, features y PRs reales.

Privacidad e integridad

Usar datos sintéticos en demostraciones. No incluir contraseñas de usuarios reales, service_role keys, tokens privados ni información personal de terceros en el repositorio público o PDF. La clave publicable del cliente no sustituye RLS. La contraseña del docente debe ser exclusiva, temporal y transmitirse por el canal autorizado por el centro.

8. Preparación del paquete final

La actividad solicita subir al LMS un único ZIP llamado APELLIDOS_NOMBRE_DI_PROYECTO.zip. Ese ZIP debe contener exclusivamente un PDF con los enlaces y evidencias. No debe contener node_modules, código fuente suelto ni varios documentos.

Lista de cierre

- Completar este PDF con URL de producción y credenciales de cuentas sintéticas de administrador y usuario estándar. Si la política del centro lo permite, compartir contraseñas temporales de demo y rotarlas después.
- Adjuntar exportación/capturas reales de Lighthouse y verificar Accessibility y Best Practices ≥ 90/100.
- Crear/mostrar un historial Git coherente con main, develop, features y Pull Requests; confirmar que el repositorio es accesible para el docente.
- Resolver o justificar la fuente NoSQL que exige la actividad.
- Verificar manualmente login, permisos RLS, formularios, fotos, responsive, teclado y exportación web/móvil.
- Regenerar Compodoc y confirmar que README/manual enlazan a la documentación vigente.
- Exportar el PDF final, comprimir solo ese archivo con el nombre exigido y abrir el ZIP para comprobar su contenido.

Repositorio de código

https://github.com/jorgeberenguer2023-ops/RetoSupabase1
La rama principal publicada se puede revisar en el enlace anterior. La actividad evalúa historial y evolución; la memoria no sustituye commits, ramas, despliegue ni evidencias auditables.

Cierre

El núcleo funcional del proyecto y la compilación Android están disponibles y verificados. Los elementos que dependen de infraestructura o evidencias externas permanecen marcados como pendientes para evitar atribuir resultados no obtenidos. Una vez completados esos puntos, actualizar esta memoria y generar el único PDF de entrega.

Fin de la memoria · Nexo · Actividad individual DI 0488